$ cn-at-scaler

Computer Networks

Network programming, from the socket up

2.5	7	2026
hours	system calls	our port

socket · bind · listen · accept · read · write · close

Week	Broad Topic
Week 1	Network programming 101
Week 2	Computer Networking Eagle Eye View
Week 3	Server Side Considerations for scale
Week 4	nginx deep dive
Week 5	Evolution of HTTP
Week 6	(Ab)using CDNs
Week 7	Economics of cloud tech
Week 8	Building for failures

the plan

2.5 hours, built bottom-up.

We start with the smallest program that can accept a TCP connection, and end with designing your own binary protocol.

A server, and everything that breaks around it

A client, plus OSI, DNS and byte order

curl, many clients at once, packets on the wire, SSL

Designing a protocol of your own

The Server

01

- A working echo server in 20 lines
- Ports, and why 1024 is a line in the sand
- The accept queue
- The signal that will kill your process

A TCP server is seven system calls.

Everything else you have ever used — Express, Flask, net/http — is a wrapper around this sequence.



the four that only a server makes

echo_server.c

Send it anything. It sends it back, once.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(2026);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 1);

    while (1) {
        int client_fd = accept(server_fd, NULL, NULL);
        char buf[4096];
        int n = read(client_fd, buf, sizeof(buf));
        write(client_fd, buf, n);
        close(client_fd);
    }
}
```

demo

Compile it. Talk to it.

Two commands and you have a network service running on your laptop.

```
$ gcc -o echo_server echo_server.c
$ nc localhost 2026
```

ports

Everything below 1024 belongs to root.

Not a rule you cannot break — a rule about who is allowed to break it.

< 1024 root only

80	443	25	21	110
HTTP	HTTPS	SMTP	FTP	POP

≥ 1024 anyone

3000	8080	6379	2026
node	alt-http	redis	ours

Why does it matter?

Binding port 80 means running as root. Running as root means a bug in your request parser is a root compromise.

That is why real deployments bind 8080 as an unprivileged user and let a front proxy — which does almost nothing — own 80 and 443.

thought experiment

What breaks if you skip a call?

Two questions worth arguing about before I give you the answer.

NO BIND()

Connection refused

Nothing owns the port, so the kernel answers the SYN with an RST straight away.

Fast, loud, easy to debug. The client knows instantly that nobody is home.

BIND() BUT NO ACCEPT()

It just hangs

The kernel completes the handshake on your behalf. The connection is established and sits in the queue.

Your process never picks it up, so the client waits until TCP times out. Silent, slow, and it looks exactly like an overloaded server.

echo_server_persistent.c

One read() is not a conversation.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(2026);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 1);

    while (1) {
        int client_fd = accept(server_fd, NULL, NULL);
        char buf[4096];
        int n;
        while ((n = read(client_fd, buf, sizeof(buf))) > 0) {
            write(client_fd, buf, n);
        }
        close(client_fd);
    }
}
```

```
listen(server_fd, 1)
```

That second argument is a queue.

The kernel accepts connections on your behalf and stacks them up. The stack has a size.

1. **The kernel finishes the handshake without you**
   By the time accept() returns, the TCP connection has been established for a while. The client has been waiting in line.

2. **Full queue behaviour is not portable**
   Connection refused on some systems; an indefinite 'Trying...' on others. Do not build logic on top of which one you saw locally.

3. **We passed 1. Real servers pass hundreds**
   And Linux caps whatever you ask for at net.core.somaxconn, so the number in your code may not be the number in effect.

SIGPIPE is the one that will kill you.

Write to a socket the peer already closed and, by default, your process dies. No error, no exception — gone.

2 SIGINT — Control-C	9 SIGKILL — cannot be caught	**13** **SIGPIPE — write to closed peer**	15 SIGTERM — the polite kill

SIGHUP on SSH disconnect · SIGSTOP / SIGTSTP on Control-Z

sigpipe_server.c

Two writes. The second one never runs.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM,0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 1);

    int client_fd = accept(server_fd,NULL, NULL);
    char buf[4096];
    int n = read(client_fd, buf, sizeof(buf));

    sleep(1); // give the client time to force-close the connection (RST)

    write(client_fd, buf, n); // peer already reset -> SIGPIPE here
    write(client_fd, buf, n); // not reached: SIGPIPE's default action kills the process

    return 0;
}
```

sigpipe_client.c

SO_LINGER {1, 0} sends RST instead of FIN.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    addr.sin_port = htons(8080);
    connect(fd, (struct sockaddr*)&addr, sizeof(addr));

    write(fd, "hello", 5);

    struct linger l = {1, 0}; // abortive close: sends RST instead of FIN
    setsockopt(fd, SOL_SOCKET, SO_LINGER, &l, sizeof(l));
    close(fd);

    return 0;
}
```

You Already Have a Client

02

- telnet, and not just to port 80
- openssl s_client for anything encrypted
- netcat for everything else

telnet is a client. Type the protocol yourself.

If a protocol is text, your keyboard is a perfectly good implementation of it.

```
$ telnet google.com 80
$ telnet localhost 2026
$ telnet mail.example.com 25
```

demo · telnet

Four lines of typing, and Google answers.

```
jeet@jitendras-air cn-at-scaler % telnet google.com 80
Trying 142.251.222.142...
Connected to google.com.
Escape character is '^]'.
GET / HTTP/1.1
Host: google.com

HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only:object-src 'none';base-uri 'self';script-src 'nonce-tlOFCqjOWGCT2By5DdHA0g' 'stric
t-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/oth
er-hp
Date: Wed, 12 Aug 2026 16:06:35 GMT
Expires: Fri, 11 Sep 2026 16:06:35 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 219
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN

<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
```

Most services do not speak HTTP.

And once TLS is in the way, telnet is useless — you need something that can do the handshake for you.

```
$ openssl s_client -connect www.hotstar.com:443
$ nc localhost 2026
$ nc -l 9000
```

Writing the Client

03

- A client in C — and what is missing from it
- Ephemeral ports
- Byte order: htons and friends
- DNS hiding inside a function call
- Where we are on the OSI stack

client.c

The client is shorter. Notice what is missing.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netdb.h>

int main() {
    struct hostent *he = gethostbyname("localhost");

    int fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    memcpy(&addr.sin_addr, he->h_addr_list[0], he->h_length);
    addr.sin_port = htons(8080);
    connect(fd, (struct sockaddr*)&addr, sizeof(addr));

    write(fd, "hello", 5);

    struct linger l = {1, 0}; // abortive close: sends RST instead of FIN
    setsockopt(fd, SOL_SOCKET, SO_LINGER, &l, sizeof(l));
    close(fd);

    return 0;
}
```

ephemeral ports

Clients never bind. The kernel picks a port for you.

You will not find a port number anywhere in that client — and yet the return traffic finds its way back.

SERVER

Binds a known port

It has to. Clients need a fixed address to aim at — that is the entire reason well-known ports exist.

bind() → listen() → accept()

CLIENT

Gets a random high port

Assigned by the kernel at connect() time from the ephemeral range — typically 32768–60999 on Linux.

Finite, and TIME_WAIT holds each one for ~60s after close. Busy clients do run out.

htons()

Your CPU and the network disagree on byte order.

htons = host to network, short. It exists because x86 stores 2026 backwards from how the wire wants it.

```
port 2026  =  0x07EA

little endian  (your x86 / ARM laptop)          EA 07

big endian     (network byte order)             07 EA

htons() swaps them.  htonl() for 4 bytes.  ntohs() / ntohl() on the way back.
```

gethostbyname()

One function call is a whole protocol.

gethostbyname() is DNS — a UDP request, a recursive resolver, and a cache — hidden behind a synchronous call that can block for seconds.

1

/etc/hosts first

Before any network traffic. This is why editing hosts overrides everything, and why a stale entry there is so hard to debug.

2

UDP query to your resolver

One small datagram to whatever is in /etc/resolv.conf. It recurses on your behalf: root → TLD → authoritative.

3

Cached for the TTL, and it blocks

Fast when warm, seconds when cold — with no timeout you control. Use getaddrinfo() in real code; gethostbyname is IPv4-only and not thread-safe.

osi

We have been writing layer 7 this whole time.

Ports are layer 4. IP addresses are layer 3. We touched all three in twenty lines and never named them.

7	**Application**	our echo protocol, HTTP
4	**Transport**	TCP · ports · htons(2026)
3	**Network**	IP · INADDR_ANY · 127.0.0.1
2	**Data link**	Ethernet, Wi-Fi · MAC addresses
1	**Physical**	copper, fibre, radio

HTTP & curl

04

- An HTTP client in C is just the client you already have
- A quick tour of the RFC
- Testing headers on purpose
- curl as a teaching instrument

An HTTP client is client.c with a string in it.

Point it at port 80, write the request text, read until close. That is the whole protocol.

```
GET / HTTP/1.1
Host: google.com

   ↑ CRLF line endings, and that blank line is mandatory
```

curl is the best HTTP teaching tool you have.

You can learn a lot about headers and SSL debugging from one command with enough v's on it.

```
$ curl google.com
$ curl -i google.com
$ curl -vv https://google.com 2>&1 | less
```

Many Clients at Once

05

- fork() per connection
- select and epoll
- Connection pools and ulimit
- Seeing the actual bytes: tcpdump, Wireshark, strace
- SSL

the debt comes due

Our server serves exactly one client.

When I wrote it I said it would support one at a time. Here are the two ways out, and both are still shipping in production today.

ONE PROCESS PER CONNECTION

fork()

Dead simple, and the OS does the scheduling for you. Each child has its own memory, so a crash is contained.

Costs a process per client. Fine for hundreds, hopeless at ten thousand.

ONE PROCESS, MANY SOCKETS

select / epoll

Ask the kernel which sockets are ready, then handle only those. No context switching, no per-client memory.

Harder to write, and one slow handler blocks everything. This is what nginx and Node do.

fork() returns twice. Read the return value.

It is a bit tricky — the same line of code runs in two processes, and the only way to tell which one you are is what came back.

0

You are the child. Handle this one client, then exit.

> 0

You are the parent. That is the child's PID. Close your copy of the fd and go back to accept().

-1

fork failed — out of processes or memory. Almost nobody checks for this.

fork_server.c

Accept, fork, and let the child do the talking.

```c
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM,  0);
    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons( 2026);
    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd,  1);

    while (1) {
        int client_fd = accept(server_fd,  NULL, NULL);

        if (fork() == 0) {
            char buf[4096];
            int n;
            while ((n = read(client_fd, buf,  sizeof(buf))) > 0) {
                write(client_fd, buf, n);
            }
            close(client_fd);
            return 0;
        }

        close(client_fd);
    }
}
```

One process. Ask the kernel who is ready.

Instead of blocking on one socket, hand the kernel your whole set and let it tell you which ones have data.

SELECT

Portable, and O(n)

Rebuild the fd set on every call; the kernel walks all of them every time.

FD_SETSIZE caps you around 1024 descriptors. Works everywhere, which is why it is still around.

EPOLL

Linux, and O(ready)

Register interest once. The kernel returns only the descriptors that actually changed.

kqueue on BSD and macOS, IOCP on Windows. This is what nginx and Node are built on.

the client side

Chrome would only open six connections to you.

Connection limits are not a browser quirk — they are the client-side version of the same resource problem.

6

Chrome's old per-host connection cap. Requests 7–10 simply queued.

ulimit -n

Your hard ceiling on open file descriptors. Sockets count. Often just 1024.

pool

Open once, reuse many times — you skip the TCP and TLS handshake every time after the first.

A socket can accept connections on any IP.

Read the man page for socket options once. Some of what is in there is a networking history lesson.

INADDR_ANY — listen on every interface

Convenient in development, and a decision worth making deliberately in production. Bind to a specific address when you mean one.

Promiscuous mode — see traffic that is not yours

On a hub, every frame reaches every port. This is how packet sniffers work, and why plaintext protocols died.

SO_REUSEADDR — the one you will actually use

Without it, restarting your server fails with 'address already in use' while the old socket sits in TIME_WAIT.

Stop guessing. Look at the bytes.

Capture it once, then read it as ASCII and as hex. Everything we have talked about is visible in there.

```
$ sudo tcpdump -ni any port 2026 -w 1.pcap
$ tcpdump -r local_capture.pcap -A
$ tcpdump -r local_capture.pcap -X
```

SSL secures TCP, not HTTP.

It sits under the application protocol, which is why the same machinery protects SSH, SMTP, database connections and anything else you put on a socket.

1. **Exchange keys and certificates**
   Asymmetric crypto to agree on a shared secret, then symmetric crypto for the data — because symmetric is dramatically faster.

2. **The certificate binds a key to an identity**
   And the CA chain is the reason you believe it. This is the part that expires at 3am and takes your site down.

3. **Pinning removes the CA from the trust equation**
   Common in mobile apps: trust this exact certificate, not whatever any CA in the world is willing to sign.

Protocol Design

06

- Message framing: fixed, delimited, length-prefixed
- Binary encodings: BCD, ASN.1, TLV
- base64 and hex for debugging
- RPC, gRPC and protobuf

Text won. Now protocols are going binary again.

HTTP proves how far human-readable can take you — and HTTP/2 is binary.

TEXT

You can read it

telnet into it, tcpdump it, eyeball the logs. That is a real and large advantage — it is most of why HTTP won.

Verbose, ambiguous at the edges, and slower to parse than it looks.

BINARY

The machine can read it faster

Compact, unambiguous, cheap to parse. No whitespace rules, no case-sensitivity arguments.

You need tooling to see anything at all — which is why hex and base64 fluency stops being optional.

How does the receiver know the message ended?

TCP gives you a byte stream, not messages. This is the single most important decision in your protocol, and there are only three answers.

1. **Fixed length — every message is exactly N bytes**
   Trivial to parse, zero ambiguity. You pay for it in padding, and you can never change your mind about N.

2. **Delimited — read until the sentinel**
   HTTP's blank line between headers and body. Simple, until the delimiter appears inside your data and you need escaping.

3. **Length prefix — send the size, then the bytes**
   Content-Length: 219. Read exactly that many and stop. Robust, binary-safe, and what most modern protocols do.

Binary coded decimal: four bits per digit.

The curious part is that the digits just show up. Convert to hex and the representation does not change.

decimal	9 8 7 6 5
nibbles	1001 1000 0111 0110 0101
hex	98 76 5F

half the size of ASCII · no conversion step · exact decimal arithmetic · still used in SIM cards, SMS and card transactions

asn.1

Write the grammar. Generate the codec.

The most powerful system I have seen — you describe the shape of your data once and the encoder and decoder are produced for you.

schema

Describe your data once, in a language-neutral grammar.

codegen

Encoder and decoder are generated. You never hand-write parsing.

1984

And still under X.509, LDAP, SNMP and the whole telecom stack today.

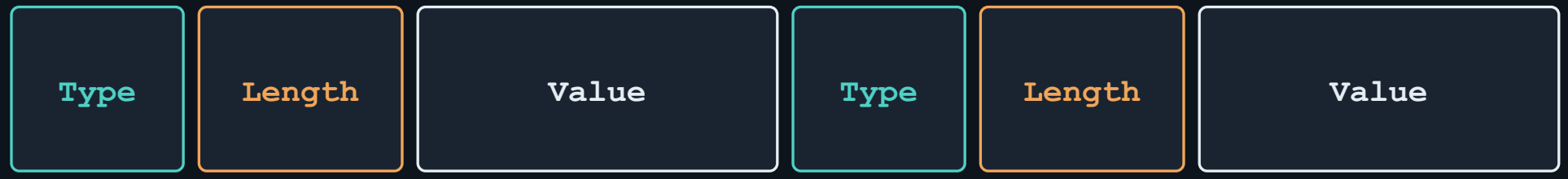
asn.1 definition

This is the whole schema. Everything else is generated.

```
MyModule DEFINITIONS ::= BEGIN
    Item ::= SEQUENCE {
        itemCode    INTEGER (1..99999),
        color       VisibleString ("Black" | "Blue" | "Brown"),
        isTaxable   BOOLEAN
    }
END
```

TLV is ASN.1 you can write on a napkin.

Type, length, value — repeated. That is the entire specification, and it is enough to build a real protocol.




```
<Type><Length><Body/Value><Type><Length></Body/Value>
```

Self-describing: a parser that has never seen this field type can still skip it, because the length tells it how far to jump.
That one property is what makes TLV forward compatible — and it is why TLS extensions, ISO 8583 and protobuf all work this way.

Hex and base64 are how you see a binary protocol.

Not fully related to framing, but base64 and hex are what you will end up using to debug any of this.

HEX

For reading

Two characters per byte, and tcpdump -X puts it beside the ASCII.

Fluency here means spotting a length prefix or a repeated tag by eye.

BASE64

For transport

Survives text-only channels: Basic auth, JWTs, email attachments, data URIs.

Costs 33% more bytes — 3 bytes become 4 characters. A wrapper, not a design.

rpc

RPC is framing plus a function name.

Send the name, send the arguments, get a response. They call it marshalling and demarshalling.

1. **Marshal — objects become bytes**
   Function name, then the arguments, encoded however your protocol says. This part is the easy part.

2. **Frame it and send it**
   Everything from the last four slides — the framing decision, the encoding decision — is now doing real work for you.

3. **The network is not a function call**
   Calls fail halfway, time out with the work possibly done, and get retried. Every RPC framework rediscovers this the hard way.

gRPC is RPC with protobuf underneath.

Much faster to encode and decode than JSON, and much easier to understand than ASN.1. That combination is why it won.

.proto

Schema first. Generate client and server code in any language.

TLV

Tag-length-value on the wire, so new fields do not break old clients.

HTTP/2

Binary and multiplexed. Many streams over one connection.

no curl

The cost: you cannot read it without tooling. Speed bought with debuggability.

Things to try

1 **Try Wireshark on your laptop**

If you use Redis, look for that traffic in Wireshark or tcpdump. RESP is text and length-prefixed — you will recognise it.

2 **Write a server and client in your language**

The calls have the same names. That is the point of having started in C.

3 **Make them do more than echo**

The moment there is structure, you have to pick a framing strategy. Notice which one you reach for.

4 **Write base64 encoding from scratch**

It forces you to think in bit groups instead of bytes — the exact shift binary protocol work needs.

5 **Pass complicated messages with TLV**

Both directions, nested structures, optional fields. Then add a field and check your old parser still works.

6 **Write a client that imitates curl**

Follow redirects, print headers, handle chunked encoding. You will end up actually reading the RFC.

$ Go break something on port 2026.

Every abstraction you use sits on the seven calls we started with. Go find the accept() inside your framework.

socket · bind · listen · accept · read · write · close